

OSPF TROUBLESHOOTING LAB

MTU Mismatches • Interface Type Mismatches • Subnet Mask Errors • Common Campus OSPF Faults

R7 ,R4 ,R2 ,Single-Area OSPF (Area 0) — R1

Platform: Cisco IOS-XE (vIOS) on EVE-NG

Lab Manual

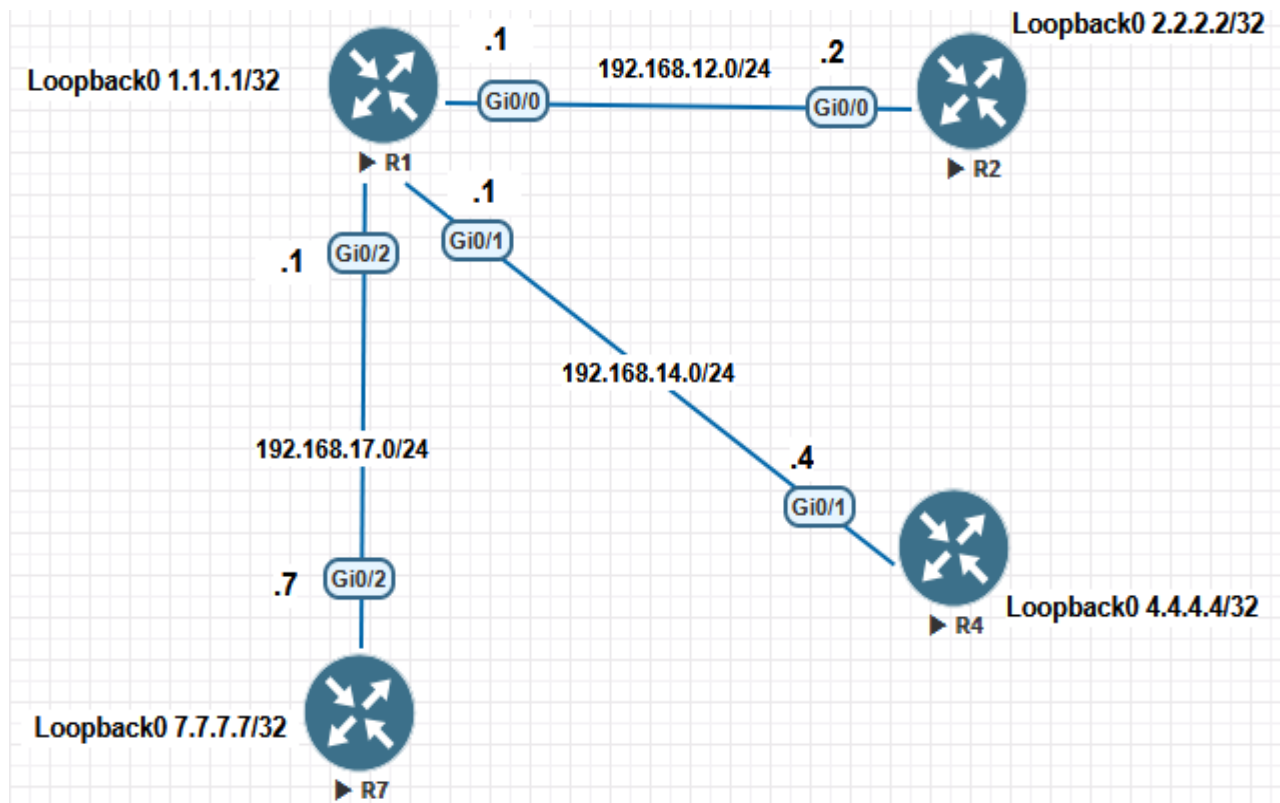


Table of Contents

Table of Contents	1
1. Introduction & Objectives	2
1.1 Learning Objectives	3
1.2 Prerequisites	3
2. Topology Overview	3
3. Baseline Configuration (Healthy OSPF)	4
3.1 R1	4
3.2 R2	5
3.3 R4	5
3.4 R7	5
4. Baseline Verification	6
4.1 Neighbor state	6
4.2 Interface / process summary	6
4.3 Routing table	6
5. Lab Scenario 1 — MTU Mismatch (R1 ↔ R2)	7
5.1 Inject the fault	8
5.2 Observe the symptom	8
5.3 Debug it	8
5.4 Fix it — two valid approaches	9
5.5 Confirm the fix	9
6. Lab Scenario 2 — OSPF Network Type Mismatch (R1 ↔ R4)	9
6.1 Inject the fault	10
6.2 Observe the symptom	10
6.3 Debug it	10
6.4 Fix it	11
6.5 Confirm the fix	11
7. Lab Scenario 3 — Subnet Mask Mismatch (R1 ↔ R7)	11
7.1 Inject the fault	12
7.2 Observe the symptom	12
7.3 Debug it	12
7.4 Fix it	12
7.5 Confirm the fix	12
8. Other OSPF Issues Common in Campus Networks	13
8.1 Area ID mismatch	14
8.2 Hello / dead timer mismatch	14

8.3 Authentication mismatch.....	14
8.4 Duplicate router-id	14
8.5 DR/BDR election problems on multi-access segments	14
8.6 Passive-interface suppressing an intended adjacency	15
8.7 Stub/NSSA area type mismatch	15
9. Limiting Debug Output — Conditional Debugging with IP Address and Interface Conditions.....	16
9.1 Baseline problem: unfiltered debug on a multi-homed router	17
9.2 Conditional debugging by interface	17
Verify and remove the condition	18
9.3 Conditional debugging by IP address.....	18
Verify and remove.....	18
9.4 Combining interface and IP address conditions	18
9.5 Applying this to the fault scenarios in this lab	19
10. Master Debugging Command Reference.....	19
11. Review Questions.....	20
Appendix A — Full Baseline Configurations.....	21
R1.....	22
R2.....	22
R4.....	22
R7.....	23

1. Introduction & Objectives

This lab uses a single-area OSPF topology (Area 0) built around R1 as the hub, with R2, R4, and R7 as spokes. The lab is split into two halves: first you will deploy a clean, fully-adjacent OSPF baseline; second you will deliberately inject three classic OSPF fault types — one at a time — and use structured debugging to identify and repair each one. A final section catalogues other OSPF faults that are common in campus networks (timers, authentication, area mismatches, DR/BDR problems, passive interfaces) with a diagnostic quick-reference table, and closes with conditional troubleshooting exercises where you are given only symptoms and must select your own diagnostic path.

1.1 Learning Objectives

- Build and verify a healthy single-area OSPF adjacency baseline across four routers.
- Recognize the show and debug output signatures of an MTU mismatch and understand why OSPF gets stuck in EXSTART/EXCHANGE.
- Recognize the symptoms of an OSPF network-type mismatch on a multi-access segment and correct it.
- Recognize the symptoms of a subnet mask mismatch between two OSPF neighbors and correct it.
- Use debug ip ospf adj, debug ip ospf hello, debug ip ospf packet, and related show commands methodically rather than by guesswork.
- Apply a repeatable troubleshooting flow to unfamiliar OSPF adjacency failures, including scenarios in campus networks with authentication, timers, and DR/BDR elections.

1.2 Prerequisites

- Comfortable with basic OSPF (LSA types, DR/BDR, hello/dead timers) — see the earlier OSPFv2 single-area and multi-area manuals in this series.
- EVE-NG lab running Cisco IOS-XE vIOS images with the topology below imported and cabled.
- Console/SSH access to R1, R2, R4, and R7.

2. Topology Overview

The topology mirrors a small campus hub-and-spoke: R1 is the distribution router with three directly connected Gigabit links, one to each of R2, R4, and R7. All links and all loopbacks are advertised into a single OSPF area — Area 0. There is no multi-area design in this lab; every fault you will inject is a link-level or interface-level fault, not an inter-area design fault.

Router	Interface	IP Address	Connects To	Subnet
R1	Loopback0	1.1.1.1/32	—	1.1.1.1/32
R1	Gi0/0	192.168.12.1/24	R2 Gi0/0	192.168.12.0/24
R1	Gi0/1	192.168.14.1/24	R4 Gi0/1	192.168.14.0/24
R1	Gi0/2	192.168.17.1/24	R7 Gi0/2	192.168.17.0/24

Router	Interface	IP Address	Connects To	Subnet
R2	Loopback0	2.2.2.2/32	—	2.2.2.2/32
R2	Gi0/0	192.168.12.2/24	R1 Gi0/0	192.168.12.0/24
R4	Loopback0	4.4.4.4/32	—	4.4.4.4/32
R4	Gi0/1	192.168.14.4/24	R1 Gi0/1	192.168.14.0/24
R7	Loopback0	7.7.7.7/32	—	7.7.7.7/32
R7	Gi0/2	192.168.17.7/24	R1 Gi0/2	192.168.17.0/24

NOTE: All Ethernet segments are point-to-point in this lab

Every link in this topology is a /24 configured on a routed GigabitEthernet interface between exactly two routers. Physically it is point-to-point even though the OSPF default network type on Ethernet is broadcast. This distinction matters directly for Lab Scenario 2.

3. Baseline Configuration (Healthy OSPF)

Configure all four routers as shown below. This is the known-good baseline you will return to — or diff against — after each fault-injection scenario. OSPF process ID 1 is used throughout; router-id is set explicitly from the loopback so it does not silently change if a physical interface flaps.

3.1 R1

```
hostname R1
!
interface Loopback0
 ip address 1.1.1.1 255.255.255.255
!
interface GigabitEthernet0/0
 ip address 192.168.12.1 255.255.255.0
 no shutdown
!
interface GigabitEthernet0/1
 ip address 192.168.14.1 255.255.255.0
 no shutdown
!
interface GigabitEthernet0/2
 ip address 192.168.17.1 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 1.1.1.1
 network 1.1.1.1 0.0.0.0 area 0
 network 192.168.12.0 0.0.0.255 area 0
```

```
network 192.168.14.0 0.0.0.255 area 0
network 192.168.17.0 0.0.0.255 area 0
```

3.2 R2

```
hostname R2
!
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
!
interface GigabitEthernet0/0
 ip address 192.168.12.2 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 2.2.2.2
 network 2.2.2.2 0.0.0.0 area 0
 network 192.168.12.0 0.0.0.255 area 0
```

3.3 R4

```
hostname R4
!
interface Loopback0
 ip address 4.4.4.4 255.255.255.255
!
interface GigabitEthernet0/1
 ip address 192.168.14.4 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 4.4.4.4
 network 4.4.4.4 0.0.0.0 area 0
 network 192.168.14.0 0.0.0.255 area 0
```

3.4 R7

```
hostname R7
!
interface Loopback0
 ip address 7.7.7.7 255.255.255.255
!
interface GigabitEthernet0/2
 ip address 192.168.17.7 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 7.7.7.7
 network 7.7.7.7 0.0.0.0 area 0
```

```
network 192.168.17.0 0.0.0.255 area 0
```

4. Baseline Verification

Before injecting any fault, confirm the baseline is fully healthy. Do this on R1 (the hub) at minimum; spot-check one spoke.

4.1 Neighbor state

```
R1# show ip ospf neighbor
```

Neighbor ID	Pri	State	Dead Time	Address	Interface
2.2.2.2	1	FULL/BDR	00:00:38	192.168.12.2	GigabitEthernet0/0
4.4.4.4	1	FULL/BDR	00:00:33	192.168.14.4	GigabitEthernet0/1
7.7.7.7	1	FULL/BDR	00:00:39	192.168.17.7	GigabitEthernet0/2

NOTE: Why every neighbor shows BDR, never DR

Each of these links has only two routers on it, so R1 and its neighbor hold a DR/BDR election even though the segment is point-to-point in practice. R1 typically wins DR on all three (higher router-id or configured priority), leaving the spoke as BDR. This is expected and harmless — it is not a fault. It becomes relevant later when you look at real DR/BDR problems on genuinely multi-access campus segments (Section 8).

4.2 Interface / process summary

```
R1# show ip ospf interface brief
```

Interface	PID	Area	IP Address/Mask	Cost	State	Nbrs	F/C
Gi0/2	1	0	192.168.17.1/24	1	BDR	1/1	
Gi0/1	1	0	192.168.14.1/24	1	BDR	1/1	
Gi0/0	1	0	192.168.12.1/24	1	BDR	1/1	
Lo0	1	0	1.1.1.1/32	1	LOOP	0/0	

4.3 Routing table

```
R1# show ip route ospf
```

```
2.0.0.0/32 is subnetted, 1 subnets
0       2.2.2.2 [110/1] via 192.168.12.2, 00:12:04, GigabitEthernet0/0
4.0.0.0/32 is subnetted, 1 subnets
0       4.4.4.4 [110/1] via 192.168.14.4, 00:12:04, GigabitEthernet0/1
7.0.0.0/32 is subnetted, 1 subnets
0       7.7.7.7 [110/1] via 192.168.17.7, 00:12:04, GigabitEthernet0/2
```

TIP: Save this baseline

Run 'copy running-config startup-config' or save an offline text copy of each router's config now. Every fault-injection scenario below tells you exactly what to change and how to revert — but having a known-good copy to diff against is the fastest way to confirm a fix is complete.

5. Lab Scenario 1 — MTU Mismatch (R1 ↔ R2)

OSPF exchanges the size of its own interface MTU inside the Database Description (DBD) packets sent during the EXSTART/EXCHANGE states. By default, IOS-XE rejects a DBD if the neighbor's advertised MTU is larger than its own — the adjacency will not proceed past EXSTART/EXCHANGE, even though hellos succeed and the neighbor briefly appears to be forming.

5.1 Inject the fault

On R2 only, lower the IP MTU on the link to R1. Leave R1 at the default 1500.

```
R2# configure terminal
R2(config)# interface GigabitEthernet0/0
R2(config-if)# ip mtu 1400
R2(config-if)# end
```

NOTE: Why this doesn't break connectivity immediately

ip mtu only affects the size IOS will use/accept before fragmenting — pings and small packets still work fine. That's exactly why MTU mismatches are sneaky in production: basic reachability tests pass, but OSPF (and other control planes sensitive to packet size) breaks.

5.2 Observe the symptom

Clear the adjacency to force renegotiation, then watch the neighbor state stall.

```
R1# clear ip ospf process
Reset ALL OSPF processes? [no]: yes

R1# show ip ospf neighbor
```

Neighbor ID	Pri	State	Dead Time	Address	Interface
2.2.2.2	1	EXSTART/BDR	00:00:35	192.168.12.2	GigabitEthernet0/0

Wait 30–60 seconds and check again — a genuine MTU mismatch will not progress past EXSTART or will flap between EXSTART and EXCHANGE; it never reaches FULL.

5.3 Debug it

```
R1# debug ip ospf adj
```



```
OSPF-1 ADJ Gi0/0: Nbr 2.2.2.2: Neighbor stuck in EXSTART/EXCHANGE, retransmit DBD
OSPF-1 ADJ Gi0/0: Mismatched dbd fields, MTU 1500 sent 1400 rcvd
```

show ip ospf interface confirms the configured MTU IOS-XE believes it should be using on that interface, and show interfaces confirms what's actually programmed on each side:

```
R1# show ip ospf interface GigabitEthernet0/0 | include MTU|ospf
Process ID 1, Router ID 1.1.1.1, Network Type BROADCAST, Cost: 1

R1# show interfaces GigabitEthernet0/0 | include MTU
MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,

R2# show interfaces GigabitEthernet0/0 | include MTU
MTU 1400 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
```

COMMON PITFALL: MTU mismatch vs. IP MTU vs. interface MTU

show interfaces reports the Layer-2 interface MTU. ip mtu is a separate, IP-layer value that can be set independently and is what OSPF DBD negotiation actually checks. On most Ethernet interfaces they track together unless ip mtu is explicitly configured, as done above — always check both when troubleshooting.

5.4 Fix it — two valid approaches

Approach A (preferred): make the MTU match on both ends. This is always the correct long-term fix in a real network.

```
R2(config)# interface GigabitEthernet0/0
R2(config-if)# no ip mtu 1400
```

Approach B (workaround, use with caution): tell OSPF to skip the MTU check entirely. This masks the mismatch rather than fixing it — real fragmentation problems for other traffic on the link will remain.

```
R1(config)# interface GigabitEthernet0/0
R1(config-if)# ip ospf mtu-ignore
R2(config)# interface GigabitEthernet0/0
R2(config-if)# ip ospf mtu-ignore
```

CAUTION: ip ospf mtu-ignore must be configured on both sides to be meaningful

Configuring it on only one router still lets that router accept a mismatched DBD, but real user traffic sized near the smaller MTU may still need to fragment or get dropped if DF-bit is set. Treat mtu-ignore as a diagnostic/emergency tool, not a permanent design choice — always prefer matching the MTU.

5.5 Confirm the fix

```
R1# show ip ospf neighbor
Neighbor ID    Pri   State           Dead Time   Address        Interface
2.2.2.2        1     FULL/BDR        00:00:32   192.168.12.2   GigabitEthernet0/0
```


6. Lab Scenario 2 — OSPF Network Type Mismatch (R1 ↔ R4)

The OSPF network type controls how hellos are sent (multicast vs. unicast), whether a DR/BDR election happens, and the hello/dead timer defaults. Two routers on the same wire must agree on network type or the adjacency will fail to form correctly — or will form but behave incorrectly.

6.1 Inject the fault

On R1 only, change the Gi0/1 network type to point-to-multipoint. Leave R4 at the Ethernet default of broadcast.

```
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ip ospf network point-to-multipoint
```

6.2 Observe the symptom

```
R1# show ip ospf neighbor
(no output for GigabitEthernet0/1 — neighbor is gone)

R4# show ip ospf neighbor
(no output for GigabitEthernet0/1 either)
```

R1 is now expecting unicast hellos to a configured list of neighbors (point-to-multipoint sends hello via 224.0.0.5 but treats the segment as a collection of point-to-point links and installs /32 host routes when adjacent — critically its hello packet's Network Mask field and timer defaults differ from broadcast). R4 is still sending broadcast-style hellos with a DR/BDR election expectation. The two sides disagree on basic hello packet contents, so the neighbor relationship that had formed is torn down.

6.3 Debug it

```
R1# debug ip ospf hello
OSPF-1 HELLO Gi0/1: Rcv hello from 4.4.4.4 area 0 192.168.14.4
OSPF-1 HELLO Gi0/1: Mismatched hello parameters from 192.168.14.4
Dead R 40 C 40 Hello R 10 C 10
```

show ip ospf interface is the most direct confirmation — it prints the network type explicitly, along with the fact that no DR/BDR is elected under point-to-multipoint:

```
R1# show ip ospf interface GigabitEthernet0/1
GigabitEthernet0/1 is up, line protocol is up
Internet Address 192.168.14.1/24, Area 0, Attached via Network Statement
Process ID 1, Router ID 1.1.1.1, Network Type POINT_TO_MULTIPOINT, Cost: 1
Transmit Delay is 1 sec, State POINT_TO_MULTIPOINT,

R4# show ip ospf interface GigabitEthernet0/1
GigabitEthernet0/1 is up, line protocol is up
Internet Address 192.168.14.4/24, Area 0, Attached via Network Statement
Process ID 1, Router ID 4.4.4.4, Network Type BROADCAST, Cost: 1
```

Transmit Delay is 1 sec, State DR, Priority 1

NOTE: Why point-to-multipoint also changes the default hello/dead timers

Broadcast networks default to a 10-second hello / 40-second dead timer. Point-to-multipoint defaults to 30-second hello / 120-second dead. Even if the network-type mismatch itself didn't block the adjacency, the resulting timer mismatch would — OSPF requires hello and dead intervals to match exactly between neighbors. This is a good example of one misconfiguration triggering a second, related mismatch automatically.

6.4 Fix it

Since this link is physically point-to-point (only R1 and R4 on the segment), the cleanest fix for a real deployment is to make both ends point-to-point — this avoids an unnecessary DR/BDR election and uses /32-free, single-cost routing. Matching both back to the Ethernet default of broadcast is equally valid and is what this lab's baseline uses.

```
! Option A - restore the baseline (broadcast, matches R4 default)
R1(config)# interface GigabitEthernet0/1
R1(config-if)# no ip ospf network point-to-multipoint

! Option B - engineer both ends as point-to-point (best practice for a routed
! point-to-point link; skips DR/BDR election entirely)
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ip ospf network point-to-point
R4(config)# interface GigabitEthernet0/1
R4(config-if)# ip ospf network point-to-point
```

6.5 Confirm the fix

```
R1# show ip ospf neighbor
Neighbor ID    Pri   State           Dead Time   Address        Interface
4.4.4.4        1    FULL/ -         00:00:33    192.168.14.4   GigabitEthernet0/1
```

Note the dash in the State column instead of DR/BDR/DROTHER — that confirms point-to-point mode is active with no election, if you chose Option B.

7. Lab Scenario 3 — Subnet Mask Mismatch (R1 ↔ R7)

OSPF hello packets carry the sending interface's subnet mask (for broadcast and non-broadcast network types). If the receiving router's own interface mask does not match what's inside the hello, the router discards the hello and logs a mismatch — the neighbor never appears at all, not even transiently.

7.1 Inject the fault

On R7 only, change the mask on Gi0/2 from /24 to /25. Leave R1 at /24.

```
R7(config)# interface GigabitEthernet0/2
R7(config-if)# ip address 192.168.17.7 255.255.255.128
```

CAUTION: This also silently changes reachability, not just OSPF

192.168.17.7/25 sits in the 192.168.17.0/25 subnet, and 192.168.17.1/24 sits in 192.168.17.0/24. These overlap at the address level so ARP may still resolve, but the routers disagree about the boundary of the subnet — exactly the kind of latent, hard-to-spot fault a mask typo produces in production.

7.2 Observe the symptom

```
R1# show ip ospf neighbor
(no entry for GigabitEthernet0/2 — R7 never appears as a neighbor)
```

7.3 Debug it

```
R1# debug ip ospf hello
OSPF-1 HELLO Gi0/2: Rcv hello from 7.7.7.7 area 0 192.168.17.7
OSPF-1 HELLO Gi0/2: Mismatched hello parameters from 192.168.17.7
Mask R 255.255.255.128 C 255.255.255.0
```

The 'Mask R ... C ...' line is the signature: R is the mask Received inside the neighbor's hello, C is this router's own Configured mask on that interface. Whenever these two differ, the hello is silently dropped.

```
R1# show ip interface GigabitEthernet0/2 | include Internet address
Internet address is 192.168.17.1/24

R7# show ip interface GigabitEthernet0/2 | include Internet address
Internet address is 192.168.17.7/25
```

7.4 Fix it

```
R7(config)# interface GigabitEthernet0/2
R7(config-if)# ip address 192.168.17.7 255.255.255.0
```

7.5 Confirm the fix

```
R1# show ip ospf neighbor
Neighbor ID      Pri   State           Dead Time   Address      Interface
7.7.7.7          1    FULL/BDR        00:00:37   192.168.17.7 GigabitEthernet0/2
```

TIP: General pattern across all three scenarios

MTU mismatch: hello succeeds, adjacency stalls in EXSTART/EXCHANGE. Network-type mismatch: hello fails or is malformed, neighbor drops or misbehaves (wrong timers, wrong election). Mask mismatch: hello is dropped outright, neighbor never appears. Which of these three 'shapes' you see is itself a diagnostic clue — use it to narrow down the cause before you even open a debug.

8. Other OSPF Issues Common in Campus Networks

The three faults above are link-level mismatches. In a larger campus network — multiple VLANs, multiple closets, shared distribution switches — several other OSPF faults show up repeatedly. This section is a diagnostic quick-reference rather than a full lab; try reproducing one or two of these yourself using the same routers. Section 9 then shows how to keep the debug output from any of these commands readable once R1 has several active OSPF links at once.

8.1 Area ID mismatch

Two interfaces on the same subnet configured into different OSPF areas will never form an adjacency.

```
R1# debug ip ospf hello
OSPF-1 HELLO Gi0/0: Mismatched hello parameters from 192.168.12.2
Area R 0.0.0.1 C 0.0.0.0
```

8.2 Hello / dead timer mismatch

Manually configured with ip ospf hello-interval / ip ospf dead-interval that no longer match the neighbor (common after copy-pasting a config between a WAN edge and a campus interface).

```
R1# debug ip ospf hello
OSPF-1 HELLO Gi0/0: Mismatched hello parameters from 192.168.12.2
Dead R 120 C 40 Hello R 30 C 10
```

8.3 Authentication mismatch

Common on campus distribution links once MD5 or SHA authentication is rolled out inconsistently — one side configured, the other not, or configured with a different key or key ID.

```
R1# debug ip ospf adj
OSPF-1 ADJ Gi0/0: Rcv pkt from 192.168.12.2: Mismatch Authentication type
```

```
%OSPF-4-ERRRCVD: OSPF: Rcv pkt from 192.168.12.2, GigabitEthernet0/0 :  
Md5 Authentication mismatch, key id 1
```

8.4 Duplicate router-id

Two routers cloned from the same golden image (or both left on the OSPF default of 'highest loopback/interface IP') can end up with identical router-ids. One neighbor relationship will refuse to come up, and LSAs may be silently overwritten in the database.

```
%OSPF-4-DUP_RTRID_NBR: Detected router with duplicate router ID 4.4.4.4  
R1# show ip ospf | include Router ID  
Router ID 4.4.4.4
```

8.5 DR/BDR election problems on multi-access segments

On a true multi-access campus VLAN (several routers/L3 switches on one broadcast domain, unlike the point-to-point links used elsewhere in this lab), an unintended ip ospf priority 0 or a stale DR that never re-elects after a reload can leave the segment with no DR, or a suboptimal DR far from the core.

```
R1# show ip ospf neighbor  
Neighbor ID      Pri   State           Dead Time   Address        Interface  
5.5.5.5          0    FULL/DROTHER    00:00:31    192.168.99.5   Vlan10  
6.6.6.6          0    FULL/DROTHER    00:00:38    192.168.99.6   Vlan10  
(no router shows DR or BDR - all priorities were set to 0)
```

8.6 Passive-interface suppressing an intended adjacency

passive-interface default combined with a forgotten no passive-interface on a link that should be forming an adjacency is one of the most common 'it was working, now it's not' campus tickets after a template-driven redeploy.

```
R1# show ip ospf interface GigabitEthernet0/0 | include Passive  
No Hellos (Passive interface)
```

8.7 Stub/NSSA area type mismatch

Not exercised in this single-area lab, but common once campus networks grow into multi-area designs: all routers attached to an area must agree on stub / totally-stubby / NSSA flags, or the area will not form correctly.

```
%OSPF-4-ERRRCVD: OSPF: Rcv pkt from 192.168.20.2, Area 0.0.0.20 :  
Options mismatch. Rcv 0x2, Expected 0x0 - E-bit mismatch
```

Issue	Symptom	Key debug/show	Fix
MTU mismatch	Stuck EXSTART/EXCHANGE	debug ip ospf adj / show ip ospf interface + show interfaces	Match MTU; mtu-ignore as last resort
Network type mismatch	Neighbor drops or wrong timers/election	debug ip ospf hello / show ip ospf interface	Match network type both ends
Mask mismatch	Neighbor never appears	debug ip ospf hello (Mask R/C)	Match subnet mask
Area mismatch	Neighbor never appears	debug ip ospf hello (Area R/C)	Match area on both interfaces
Timer mismatch	Neighbor never appears / flaps	debug ip ospf hello (Dead/Hello R/C)	Match hello & dead intervals
Auth mismatch	Neighbor never appears	debug ip ospf adj (auth error)	Match auth type, key id, key
Duplicate router-id	One adjacency won't form	%OSPF-4-DUP_RTRID_NBR	Set unique router-id, clear process
DR/BDR problems	No DR on multi-access segment	show ip ospf neighbor (all DROTHER)	Fix priority, or clear + reconverge
Passive-interface	No hellos sent at all	show ip ospf interface (Passive)	no passive-interface on that link
Stub/NSSA mismatch	Area won't form (multi-area)	debug ip ospf adj (Options/E-bit)	Align stub/NSSA flags area-wide

9. Limiting Debug Output — Conditional Debugging with IP Address and Interface Conditions

R1 has three simultaneous OSPF adjacencies. A plain debug ip ospf hello prints a hello event for every interface roughly every 10 seconds — on a hub router that quickly becomes an unreadable wall of interleaved text, especially over a slow SSH session. IOS has a dedicated feature for this called conditional debugging: you define one or more conditions (an interface, or a specific neighbor IP address), and every subsequent debug you enable is automatically filtered down to only the events that match those conditions.

NOTE: This is different from 'debug ip ospf hello' with no filtering

Conditional debugging (the debug condition family of commands) is a global filter that sits in front of every debug facility on the router, not an option of the debug ip ospf commands themselves. Once at least one condition is active, ALL debug output system-wide — not just OSPF — is restricted to messages associated with that condition. This is a router-wide switch, so remove conditions with undebug condition all when you're done.

9.1 Baseline problem: unfiltered debug on a multi-homed router

With no conditions set, enabling a hello debug on R1 mixes together events from all three links, making it hard to isolate the one you actually care about (in this example, the R1↔R4 link):

```
R1# debug ip ospf hello
OSPF-1 HELLO Gi0/0: Send hello to 224.0.0.5 area 0
OSPF-1 HELLO Gi0/1: Send hello to 224.0.0.5 area 0
OSPF-1 HELLO Gi0/2: Send hello to 224.0.0.5 area 0
OSPF-1 HELLO Gi0/0: Rcv hello from 2.2.2.2 area 0 192.168.12.2
OSPF-1 HELLO Gi0/2: Rcv hello from 7.7.7.7 area 0 192.168.17.7
OSPF-1 HELLO Gi0/1: Rcv hello from 4.4.4.4 area 0 192.168.14.4
(repeats for every interface, every 10 seconds, until undebug all)
```

9.2 Conditional debugging by interface

debug condition interface restricts all subsequent debug output to events tied to a single interface. Use this when you already know which link is misbehaving — for example, isolating the R1↔R4 fault from Section 6 without any noise from Gi0/0 or Gi0/2.

```
R1# debug condition interface GigabitEthernet0/1
Condition 1 set

R1# debug ip ospf hello
OSPF-1 HELLO Gi0/1: Send hello to 224.0.0.5 area 0
OSPF-1 HELLO Gi0/1: Rcv hello from 4.4.4.4 area 0 192.168.14.4
OSPF-1 HELLO Gi0/1: Mismatched hello parameters from 192.168.14.4
Dead R 120 C 40 Hello R 30 C 10
```

Notice Gi0/0 and Gi0/2 no longer print anything at all, even though their hellos are still being sent and received in the background — the condition simply stops the router from showing you events outside the interface you asked for.

Verify and remove the condition

```
R1# show debug condition
Condition 1: interface Gi0/1 (1 flags triggered)
      Flags: Ipenabled

R1# undebug condition interface GigabitEthernet0/1
Condition 1 has been removed

R1# undebug condition all
All conditions have been removed
```

CAUTION: Platform difference: no 'standard' keyword on IOS-XE

Classic IOS (12.x) supports debug condition standard <acl>, tying the filter to a standard access-list. IOS-XE — what these vIOS images run — dropped that keyword. Run debug condition ? on your own router before following along below; if standard is missing and ip is present, use the ip address form shown here instead.

9.3 Conditional debugging by IP address

On IOS-XE, debug condition ip address ties the filter directly to a neighbor's IP address — no access-list required. This is the better choice when you want to follow a specific neighbor regardless of which local interface it's reached on, useful for adjacency-level debugging (debug ip ospf adj) where the interesting field is the neighbor's address, not the local interface name.

```
R1# debug condition ip 192.168.14.4
Condition 1 set

R1# debug ip ospf adj
OSPF-1 ADJ  Gi0/1: Nbr 4.4.4.4: Neighbor stuck in EXSTART/EXCHANGE, retransmit DBD
OSPF-1 ADJ  Gi0/1: Mismatched dbd fields, MTU 1500 sent 1400 rcvd
```

Output tied to R2 (2.2.2.2) or R7 (7.7.7.7) is suppressed entirely, even though debug ip ospf adj is enabled process-wide. Add a second debug condition ip address line for each additional neighbor you want to follow — conditions of the same type are ORed together, so multiple ip address conditions widen coverage rather than narrowing it (see 9.4 for what happens when you mix different condition types).

Verify and remove

```
R1# show debug condition
Condition 1: IP Address 192.168.14.4 (1 flags triggered)
      Flags: Ipenabled

R1# undebug condition ip 192.168.14.4
Condition 1 has been removed
```

9.4 Combining interface and IP address conditions

Conditions of the same type (e.g. two ip address entries) are ORed together — either address triggers output. Conditions of different types (an interface condition plus an ip address condition) are ANDed together — a message must satisfy both to print. This lets you narrow further, for example to only the R4 neighbor's traffic as seen specifically on Gi0/1:

```
R1# debug condition interface GigabitEthernet0/1
Condition 1 set
R1# debug condition ip 192.168.14.4
Condition 2 set

R1# show debug condition
Condition 1: interface Gi0/1 (1 flags triggered)
Condition 2: IP Address 192.168.14.4 (1 flags triggered)
```

CAUTION: Same-type = OR, different-type = AND

Two debug condition ip address entries widen coverage (OR) — useful for following two neighbors at once. An interface condition plus an ip address condition narrows coverage (AND) — useful for pinning down one neighbor on one specific link. Run show debug condition and read it carefully before trusting the filtered output; it's easy to assume the wrong logic and conclude a fault is 'gone' when it's just being filtered out.

9.5 Applying this to the fault scenarios in this lab

Revisit Section 5 (MTU) and Section 7 (mask mismatch) with conditional debugging active, so the console only shows the link under test:

```
! MTU scenario (Section 5), isolated to R1<->R2 only
R1# debug condition interface GigabitEthernet0/0
R1# debug ip ospf adj

! Mask scenario (Section 7), isolated to the R7 neighbor by address
R1# debug condition ip 192.168.17.7
R1# debug ip ospf hello
```

TIP: Cleanup checklist after any debugging session

- undebg all — stop the debug facilities themselves (hello, adj, packet, events).
- undebg condition all — remove any interface or ip address conditions; otherwise they silently persist and filter unrelated future debugs.

NOTE: Console flood vs. remote sessions

If you're troubleshooting over SSH/Telnet rather than the console port, remember debug output is not shown on a vty session until you run terminal monitor. Conditional debugging reduces volume; terminal monitor is what actually lets a remote session see it in the first place.

10. Master Debugging Command Reference

A consolidated reference for every command used across this lab, grouped by what question it answers. Turn off debugs with `undebug all` (or `no debug ip ospf <topic>`) as soon as you have the information you need — `debug ip ospf packet` in particular is verbose and CPU-intensive on a busy router.

Command	Answers
<code>show ip ospf neighbor</code>	Is the neighbor up, and in what state / role (DR, BDR, DROTHER)?
<code>show ip ospf interface [brief]</code>	Area, network type, cost, timers, DR/BDR, passive status per interface
<code>show ip ospf interface <intf></code>	Full detail for one interface: MTU-sensitive fields, network type, state
<code>show ip interface <intf></code>	Actual configured IP address and mask on the interface
<code>show interfaces <intf></code>	Layer 2 MTU, line protocol, error counters (CRC, input/output errors)
<code>show ip route ospf</code>	Which OSPF routes actually installed — confirms end-to-end effect of a fix
<code>show ip ospf database</code>	LSA-level view; useful once adjacencies are up but routes look wrong
<code>show ip ospf</code>	Process-level summary: router-id, SPF stats, area list
<code>debug ip ospf hello</code>	Live hello packet exchange; shows Area/Mask/Dead/Hello Received-vs-Configured on any mismatch
<code>debug ip ospf adj</code>	Adjacency state-machine transitions; MTU/DBD problems, retransmissions, auth errors
<code>debug ip ospf packet</code>	Raw packet-level detail for all OSPF packet types — verbose, last resort
<code>debug ip ospf events</code>	High-level event log across hello, adjacency, SPF, and LSA flooding
<code>clear ip ospf process</code>	Force full OSPF re-negotiation — use after any fix to confirm clean reconvergence
<code>debug condition interface <intf></code>	Restrict all subsequent debug output to one interface (Section 9)
<code>debug condition ip <ip address ></code>	Restrict all subsequent debug output to one neighbor IP (Section 9)
<code>show debug condition</code>	List active conditional-debug filters and their trigger counts
<code>undebug condition all</code>	Remove all conditional-debug filters (does not stop the debugs themselves)
<code>terminal monitor</code>	Show debug/log output on the current SSH/Telnet session (not needed on console)
<code>undebug all</code>	Stop all active debugs

11. Review Questions

1. Why can two OSPF routers successfully exchange hello packets and still fail to reach the FULL state due to an MTU mismatch?
2. What is the difference between the Layer 2 interface MTU (show interfaces) and the ip mtu value OSPF actually checks during DBD exchange?
3. Name the two valid fixes for an MTU mismatch, and explain why one is preferred over the other in production.
4. On a point-to-point Ethernet link between exactly two routers, why might an engineer deliberately configure ip ospf network point-to-point rather than leaving the broadcast default?
5. What two hello-packet-derived defaults change automatically when you switch a network type from broadcast to point-to-multipoint?
6. In the debug ip ospf hello output 'Mask R 255.255.255.128 C 255.255.255.0', which value belongs to the local router and which belongs to the remote neighbor?
7. Why does a subnet mask mismatch prevent the neighbor from appearing at all, whereas an MTU mismatch allows the neighbor to appear but not reach FULL?
8. List at least four distinct root causes that can all produce the identical debug ip ospf hello message 'Mismatched hello parameters'. How do you distinguish between them?
9. Why is FULL/DROTHER — stuck at 2-WAY with a non-DR/BDR peer — sometimes the correct, expected end state rather than a fault?
10. A colleague proposes fixing an MTU mismatch by configuring ip ospf mtu-ignore on both routers permanently instead of matching the MTU. What operational risk does this leave unresolved?
11. Why might duplicate router-ids be more likely in an environment where router-id is never statically configured?
12. Why is debug condition standard <acl> not available on IOS-XE, and what command replaces it for filtering debug output by neighbor IP address?
13. Why is debug ip ospf packet described as a 'last resort' compared to debug ip ospf hello or debug ip ospf adj?
14. In a multi-area campus design (not used in this lab), what additional class of mismatch becomes possible that cannot occur in a single-area Area 0 design like this one?
15. After applying any fix in this lab, why is clear ip ospf process recommended before declaring the fix complete, rather than just waiting for the next hello?

Appendix A — Full Baseline Configurations

Complete, known-good configuration for all four routers, reproduced here for quick copy-paste after completing the fault-injection scenarios.

R1

```
hostname R1
!
interface Loopback0
 ip address 1.1.1.1 255.255.255.255
!
interface GigabitEthernet0/0
 ip address 192.168.12.1 255.255.255.0
 no ip ospf mtu-ignore
 no shutdown
!
interface GigabitEthernet0/1
 ip address 192.168.14.1 255.255.255.0
 no ip ospf network point-to-multipoint
 no shutdown
!
interface GigabitEthernet0/2
 ip address 192.168.17.1 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 1.1.1.1
 network 1.1.1.1 0.0.0.0 area 0
 network 192.168.12.0 0.0.0.255 area 0
 network 192.168.14.0 0.0.0.255 area 0
 network 192.168.17.0 0.0.0.255 area 0
```

R2

```
hostname R2
!
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
!
interface GigabitEthernet0/0
 ip address 192.168.12.2 255.255.255.0
 no ip mtu 1400
 no shutdown
!
router ospf 1
 router-id 2.2.2.2
 network 2.2.2.2 0.0.0.0 area 0
 network 192.168.12.0 0.0.0.255 area 0
```

R4

```
hostname R4
!
interface Loopback0
 ip address 4.4.4.4 255.255.255.255
!
interface GigabitEthernet0/1
 ip address 192.168.14.4 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 4.4.4.4
 network 4.4.4.4 0.0.0.0 area 0
 network 192.168.14.0 0.0.0.255 area 0
```

R7

```
hostname R7
!
interface Loopback0
 ip address 7.7.7.7 255.255.255.255
!
interface GigabitEthernet0/2
 ip address 192.168.17.7 255.255.255.0
 no shutdown
!
router ospf 1
 router-id 7.7.7.7
 network 7.7.7.7 0.0.0.0 area 0
 network 192.168.17.0 0.0.0.255 area 0
```

TIP: End of manual

Reset all four routers to this appendix configuration between practice attempts so each fault-injection scenario and conditional exercise starts from the same known-good baseline.